Overview of Generative AI

Part 1: Principles, Autoencoders, and VAEs

Day 1 - Session 3

KavinSchool

What We'll Cover

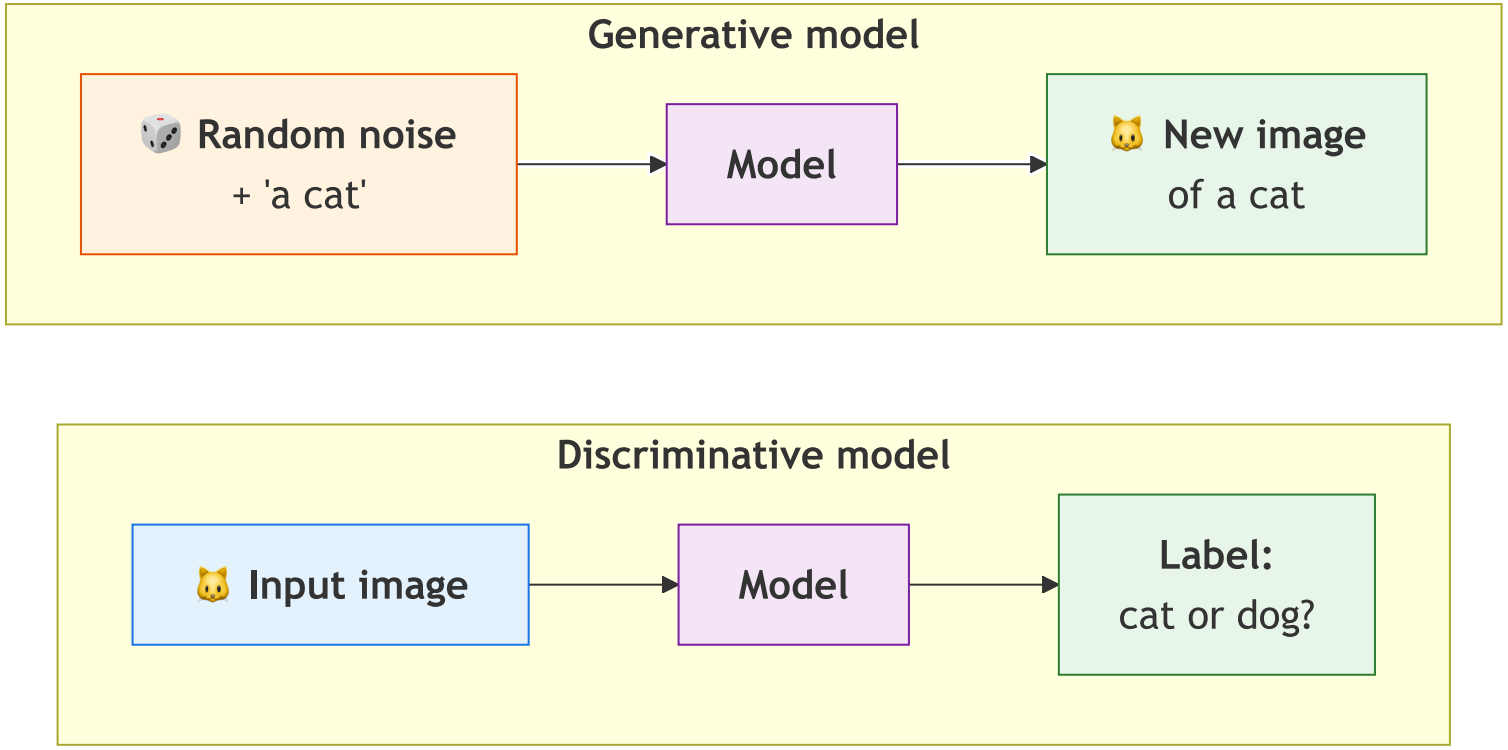
- What generative AI is and where it is used
- How generative models differ from the models we have seen
- Demo: sampling from a probability distribution
- Autoencoders and variational autoencoders
- Lab 1.3: generate handwritten digits with a VAE

What Is Generative AI?

Models that create new data

KavinSchool

Discriminative vs Generative



Discriminative models answer "which class?" Generative models answer "what does a realistic sample look like?"

In Practice: Discriminative vs Generative

Discriminative Models (Categorize)

- A shipping company needs to route support tickets. The model reads the text and tags it as "Billing", "Tracking", or "Returns". It sorts existing information into buckets.

Generative Models (Create)

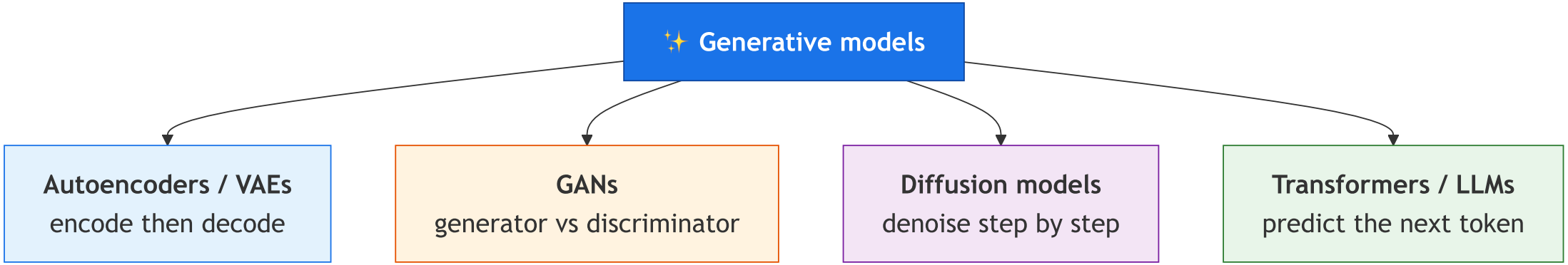
- A support agent needs a first draft response to a complaint. The model reads the ticket and writes a brand-new apology and resolution email. It creates new text from a learned pattern.

Generative AI Applications

Domain	Example
Text	Chat assistants, summarization, code
Images	Product mockups, art, editing
Audio	Speech synthesis, music
Data	Synthetic data for testing and privacy

Today we focus on the image side, which makes the ideas easy to see.

The Generative Family



Different recipes, same goal: learn a data distribution well enough to draw new samples from it. GAN: generative adversarial network. LLM: large language model.

Generative Adversarial Networks (GANs)

A GAN trains two networks that compete:

- **Generator:** creates fake samples from random noise
- **Discriminator:** tries to tell real samples from fake ones

As each improves, the generator learns to produce convincing data. GANs gave sharp image results but can be tricky to train. Today we focus on VAEs and diffusion.

The Core Idea: Sampling

Generation is sampling from a learned distribution

KavinSchool

Learn a Distribution, Then Sample

A generative model learns the probability distribution of the training data.

- Fit the shape of the data (what is likely, what is not)
- Draw a random sample from that shape
- Each sample is new but looks like it belongs

The randomness is why you get variety instead of one fixed answer.

> **IMPORTANT**
>
> Every generative model reduces to this: learn the shape of the data, then draw a fresh sample from it.

In Practice: Probabilistic Sampling

Think of a bakery that sells chocolate chip cookies.

- **The Distribution:** The baker learns the recipe (flour, sugar, butter ratio). This is the pattern.
- **Sampling:** Every time the baker makes a new cookie, it follows the recipe. No two cookies have the exact same shape or chip placement, but they all look and taste like chocolate chip cookies.
- **Enterprise View:** A retailer models customer traffic patterns (the distribution), then runs thousands of simulations (sampling) to predict checkout line lengths.

Demo: Probabilistic Sampling

Run `day1/demos/demo-probabilistic-sampling.py`.

- Fit a simple distribution to a small dataset
- Draw new samples and plot them next to the originals
- New points are not copies, yet they follow the same pattern

> This is the whole idea behind generation, shown in a few lines.

Autoencoders

Compress, then reconstruct

KavinSchool

The Autoencoder



An encoder squeezes the input into a small latent code, and a decoder rebuilds the input from that code.

Why the Bottleneck Helps

The small latent code forces the network to keep only what matters.

- It cannot copy the input, so it learns a compact representation
- Similar inputs land near each other in latent space
- This learned representation is useful on its own

A plain autoencoder reconstructs well but is not built for generating new samples.

> **TIP**
>
> Reach for a plain autoencoder to compress or denoise. Reach for a VAE when you need to generate new samples.

In Practice: Autoencoders

An autoencoder is a compressor. It forces data through a tiny pipe.

- **Fraud Detection:** A bank trains an autoencoder on normal credit card swipes. When a normal swipe goes through the pipe, it reconstructs perfectly.
- **The Anomaly:** When a stolen card is used in a strange location, the transaction does not fit the learned pattern. It reconstructs poorly. The high error score triggers a fraud alert.

Intuit Scenario: Credit Karma and QuickBooks

Intuit uses these same ideas to protect members and their data.

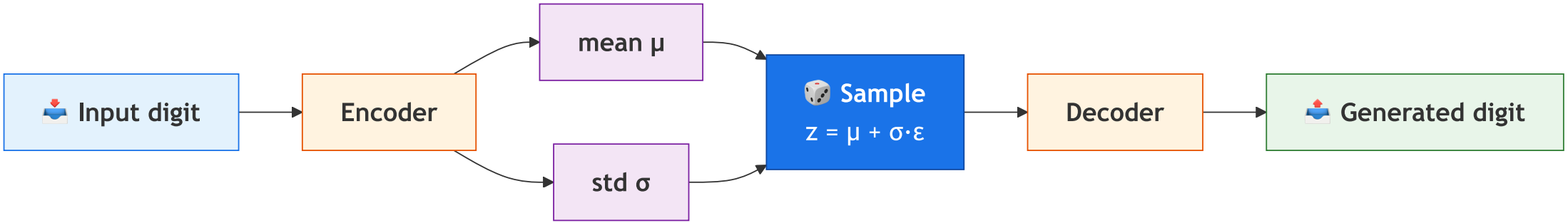
- **Anomaly detection (autoencoder):** Credit Karma learns a member's normal account activity, then flags a sign-in or transaction it reconstructs poorly for review
- **Synthetic data (VAE):** teams train on real financial records, then generate synthetic customers to test QuickBooks features without touching anyone's private information

Variational Autoencoders

An autoencoder you can sample from

KavinSchool

From Autoencoder to VAE



A VAE encodes each input to a mean and spread, then samples a point before decoding.

What the VAE Adds

- The encoder outputs a distribution (mean and standard deviation), not a single point
- Training keeps the latent space smooth and centered
- To generate, sample a random point and decode it into a new image

That smooth, organized latent space is what makes sampling produce realistic results.

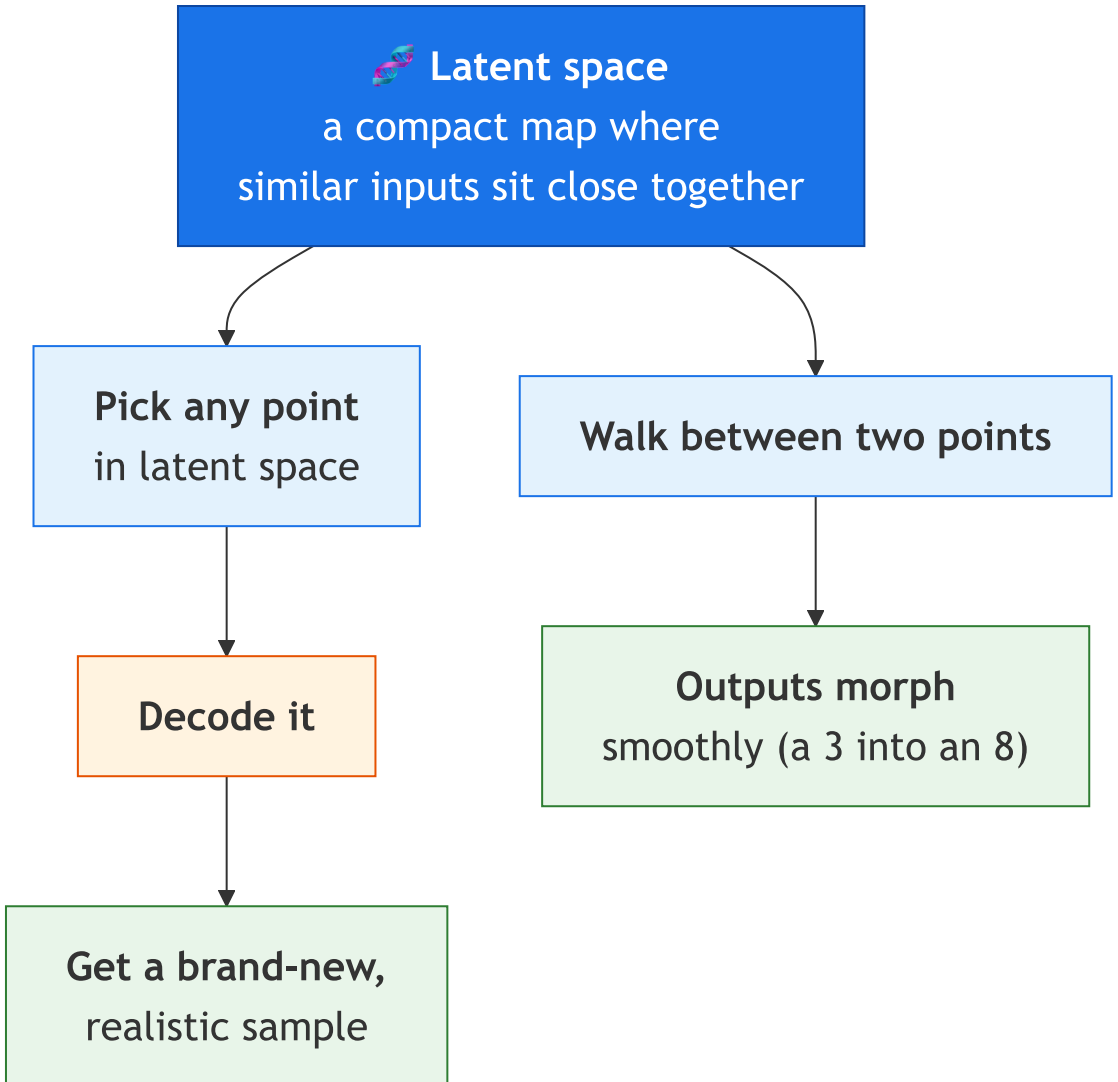
> **NOTE**
>
> The key change from a plain autoencoder: the encoder outputs a distribution (a mean and a spread), not a single point.

In Practice: VAEs

A VAE builds an organized map of possibilities.

- **Synthetic Data:** A hospital needs to share patient records with software developers, but privacy laws forbid sharing real names and histories.
- **The Solution:** They train a VAE on real patient data. The VAE builds a map of what realistic health records look like. The developers sample random points from the map to generate synthetic patients that match the real statistics but belong to nobody.

Walking the Latent Space



In Practice: Walking the Latent Space

Moving across the latent space means slowly changing the generated output.

- **Product Design:** An automotive designer generates a sleek sports car design and a boxy SUV design.
- **The Walk:** The designer asks the model to step halfway between the two points. The model generates a crossover vehicle blending both styles seamlessly.

Breakout Lab 1.3

Generate handwritten digits with a VAE

KavinSchool

Lab 1.3: A VAE for Digits

Open `day1/lab1_3_vae/start.ipynb` in Colab.

- Load the MNIST handwritten digits
- Build a small VAE encoder and decoder
- Train for a few epochs on CPU
- Sample the latent space to generate brand-new digits

> Stretch goal: interpolate between two digits, or condition on a chosen digit.

Key Takeaways

1. Generative models learn a data distribution and sample from it
2. Sampling is what turns a learned pattern into new, varied output
3. An autoencoder compresses to a latent code, then reconstructs
4. A VAE organizes the latent space so you can sample realistic data
5. You will generate new digits from noise in Lab 1.3

Questions?

Next: Diffusion Models

Evening break: 15 minutes

KavinSchool